

# Advanced NLP

## Session 02 · Attention I

Prof. Dr. Richard Sieg · TH Köln IWS · WS 26 / 27  
Tuesday, October 13, 2026

# One hour of attention, then lunch

Time

- 11:15 From RNNs to attention
- 11:40 Self-attention: queries, keys, values
- 12:15 Lunch. After lunch, you build it in notebook 02

Now



Tue 13

01 Kick-off

02 Attention I

Lunch

03 Transformer

04 Generation

Wed 14

05 Pretraining

06 Story of LLMs

Lunch

07 Post-training

08 Evaluation

Thu 15

Hackathon

Lunch

Presentations

9:30

17:00

# Attention Is All You Need

## Attention Is All You Need

---

**Ashish Vaswani\***  
Google Brain  
avaswani@google.com

**Noam Shazeer\***  
Google Brain  
noam@google.com

**Niki Parmar\***  
Google Research  
nikip@google.com

**Jakob Uszkoreit\***  
Google Research  
usz@google.com

**Llion Jones\***  
Google Research  
llion@google.com

**Aidan N. Gomez\*<sup>†</sup>**  
University of Toronto  
aidan@cs.toronto.edu

**Łukasz Kaiser\***  
Google Brain  
lukaszkaiser@google.com

**Illia Polosukhin\*<sup>‡</sup>**  
illia.polosukhin@gmail.com

# The Illustrated Transformer

by Jay Alammar

# Agenda

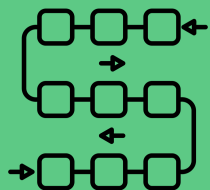
01.

Attention

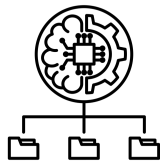
02.

Self-attention

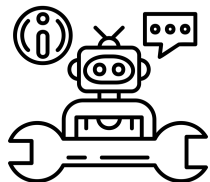
# Three ingredients make an LLM



Process long sequences and context



Efficient training on huge datasets



Follow (human) instructions

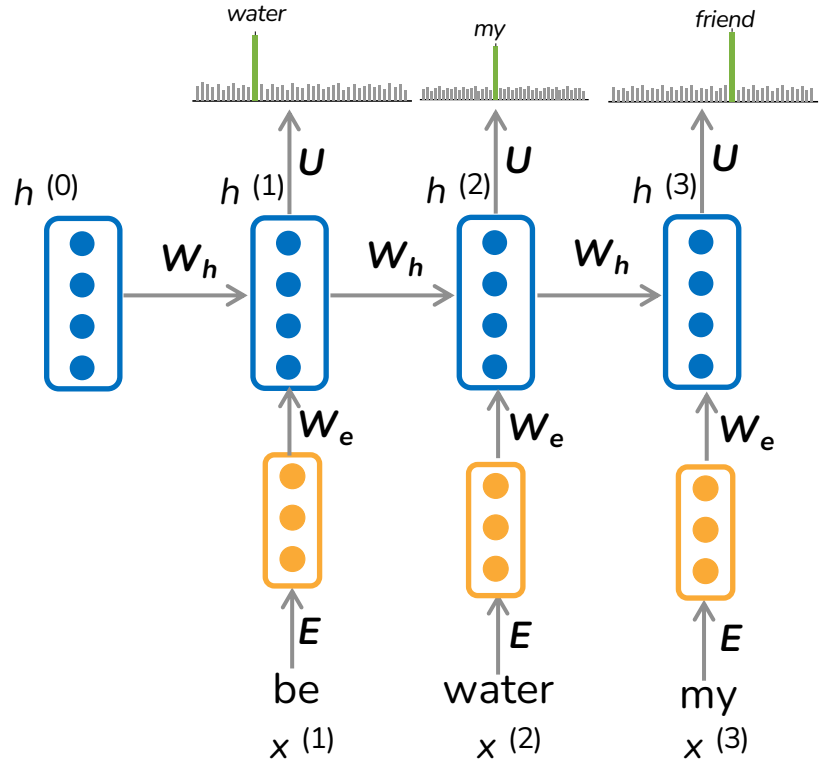
Attention computes the company  
a word keeps.

01.

Attention

# An RNN carries its memory along

- + We can process sequences of any length
- + Size of the model does not depend on input size
- + Next word prediction can theoretically use information from the whole sequence
- Computation is very slow
- Information from many steps back gets lost easily



# Information fades over many steps

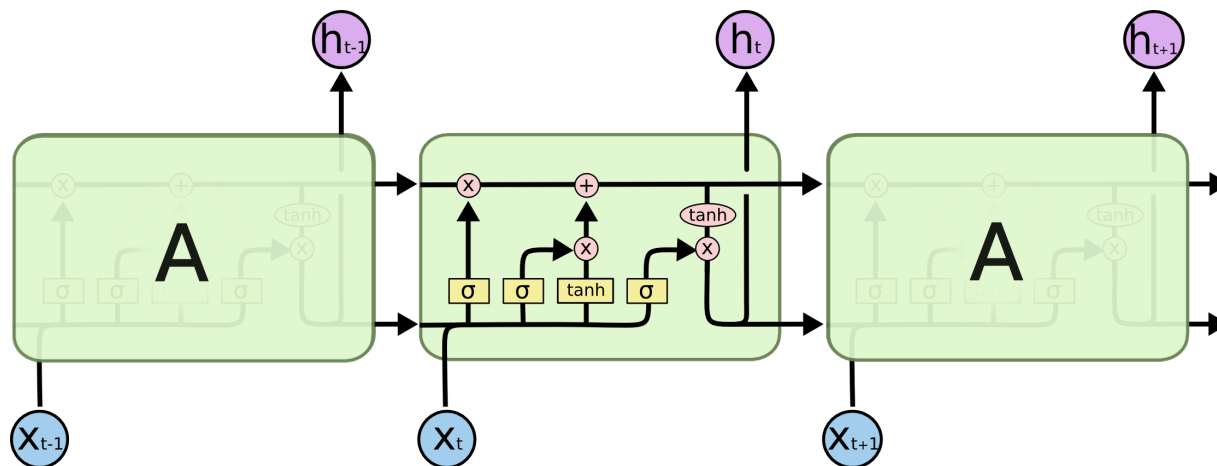
- “When she tried to print her **tickets**, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her \_\_\_\_\_”
- The word tickets is very far away from the target word in the end
- If the gradient is too small, the model cannot learn this dependency

## Can we fix this?

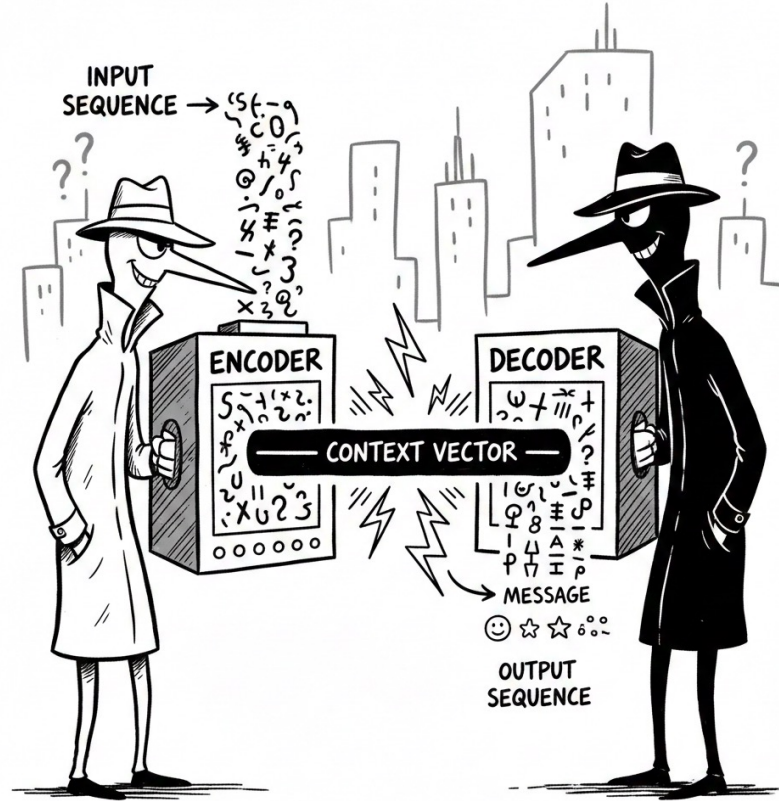
- RNN vanilla architecture makes it too difficult to preserve information over many steps since the hidden state is constantly being rewritten
- We need some kind of separate **memory** (LSTMs)
- Or a direct passthrough where all words refer to all other words (**attention**)

# LSTMs kept the memory alive longer

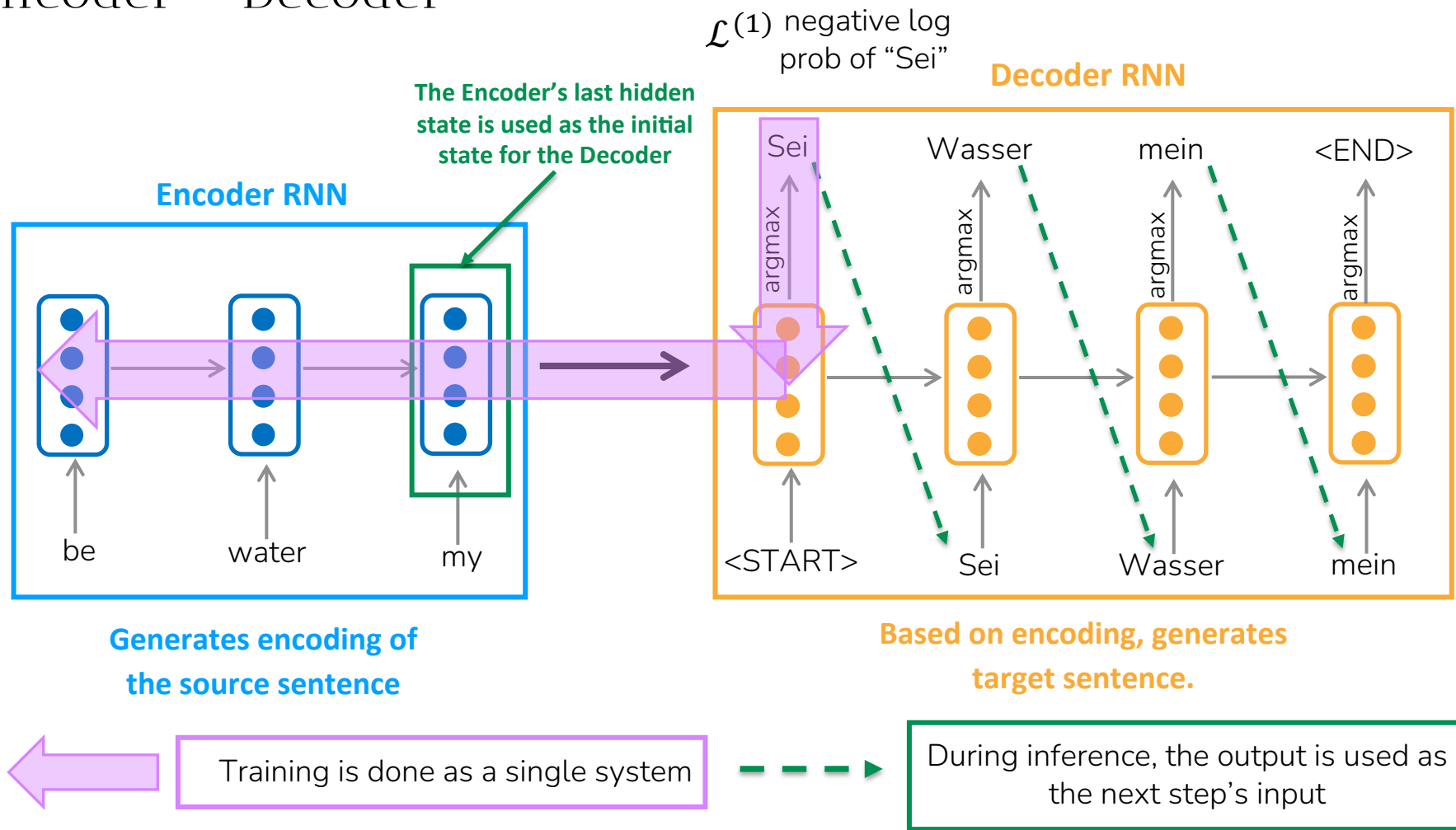
- LSTMs mitigate the vanishing gradient issue of RNNs, memory can be preserved
- Allows us to process much longer sequences
- From 2013 up until the **transformer architecture** in 2017, LSTMs were the dominant NLP approach and achieved state-of-the-art results (e.g. machine translation, image captioning, language models)
- 2016 Google switched to LSTMs for machine translation



# Encoder - Decoder



# Encoder - Decoder



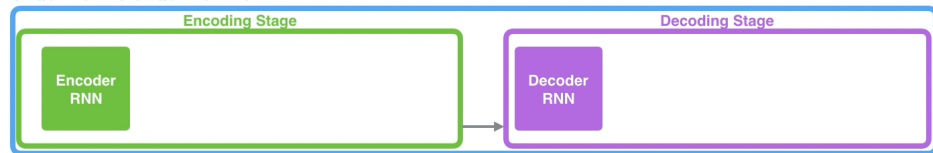
# One hidden state cannot carry a whole sentence

- The entire meaning of the source sentence is encapsulated in the last hidden state
- The decoder only uses this last hidden state but has no other interaction with the encoder
- Words are processed in sequential order and thus it is difficult to learn long-distance dependencies
- We have to compute the hidden states sequentially so we cannot parallelize and use GPUs for training and inference

# An attention decoder looks back at every encoder state

## Neural Machine Translation

### SEQUENCE TO SEQUENCE MODEL



Je            suis            étudiant

## Neural Machine Translation

### SEQUENCE TO SEQUENCE MODEL WITH ATTENTION

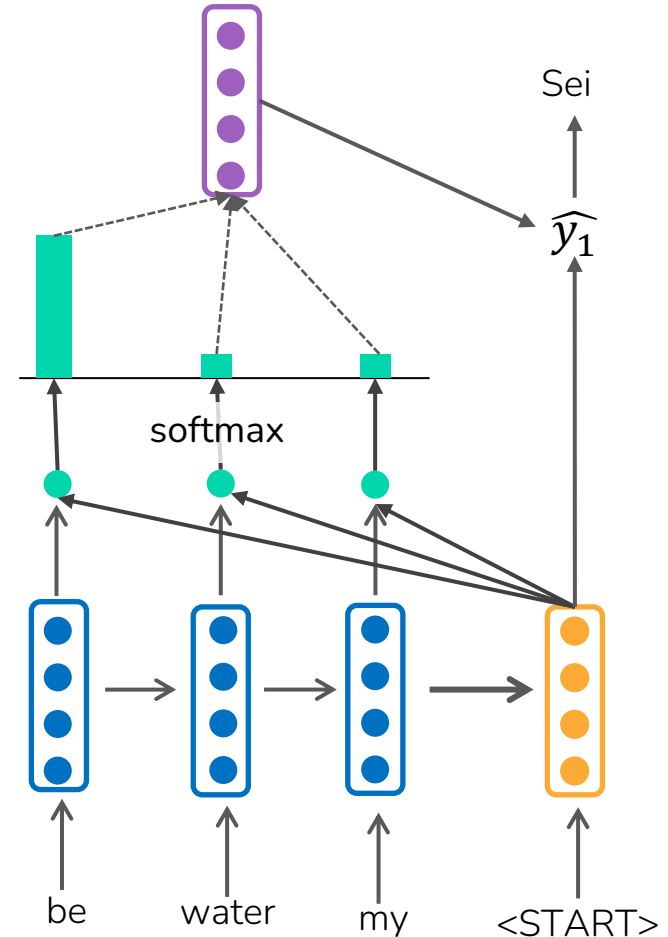


Je            suis            étudiant

Attention Decoder: Bahdanau et al. 2014,  
Luong et al. 2015

# Attention is a weighted average

- At each time step  $t$  in the decoder, we take the average of all encoder hidden states as an additional **context vector** combined with the hidden state
- Instead of just taking the normal average of all hidden states we take a **weighted average** depending on the current hidden state
- Weights are given by softmaxed **attention scores** so that they sum up to 1
- And these attention scores are learned by the neural network



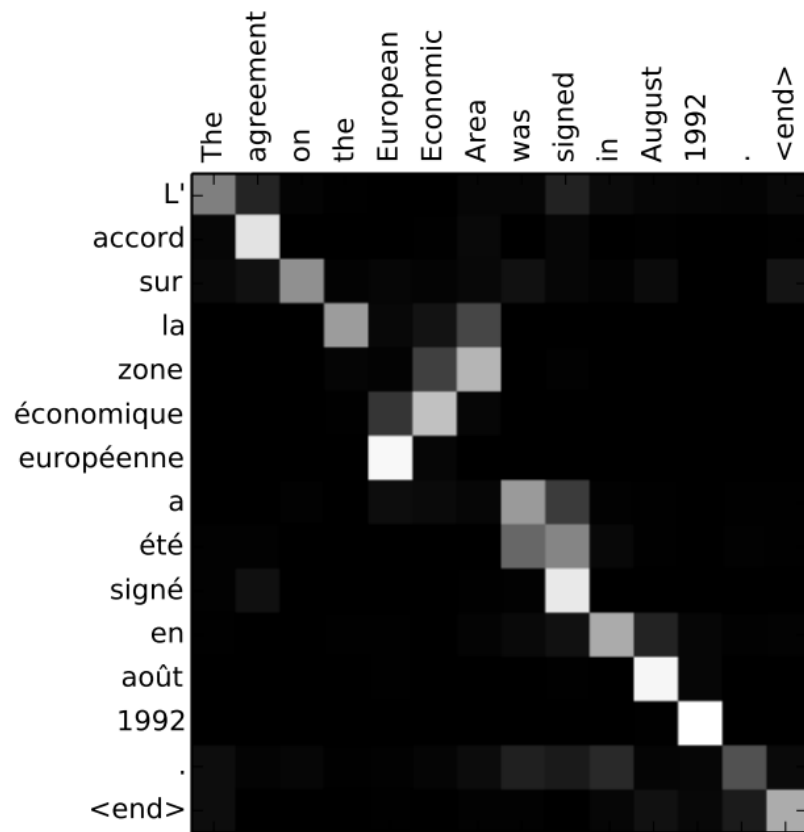
# At every step, the decoder builds a fresh context vector

1. The **Attention Decoder** receives **all** hidden states of the Encoder.
2. At each time step, we compute a score for each encoder hidden state (**attention**)
3. We multiply each hidden state with its **softmaxed** score
4. And add those vectors to get our new **context vector**
5. The context vector is then joined with the hidden state vector to generate the output distribution

Attention at time step 4

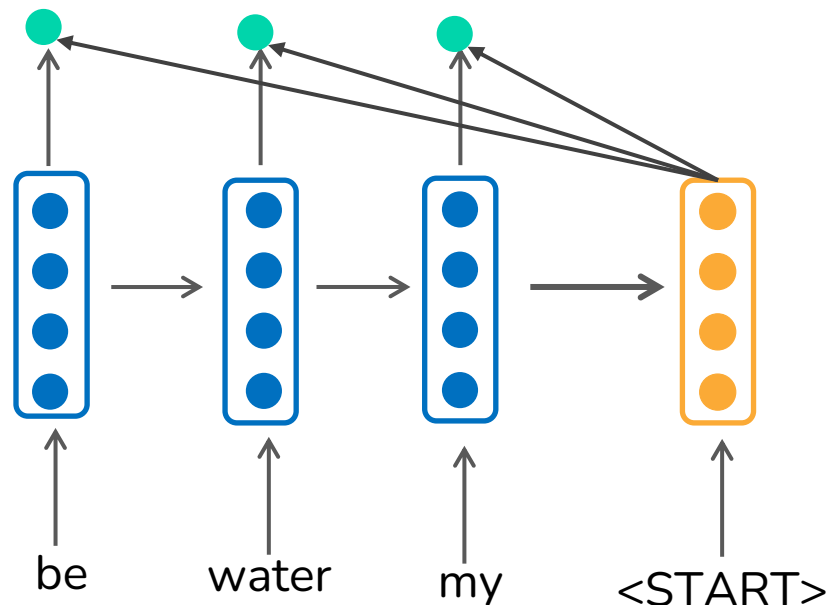


# Attention learns which source words each output word needs



# Computing Attention Scores

- Easiest approach: Take the scalar/dot product between encoder and decoder hidden state  $h_E \cdot h_D$
- More generally: Introduce a weight matrix:  $h_E^T W h_d$
- Use a feed forward layer and project it to a scalar:  $v^T \tanh(W_1 h_E + W_2 h_D)$



# Attention fixed the bottleneck, but the RNN is still there

- The encoder and decoder work sequentially and we cannot parallelize computations
- The attention decoder now has all encoder hidden states but only the current decoder hidden state to work with
- Vanishing gradients: difficult to learn long-distance relationships

As it turns out:

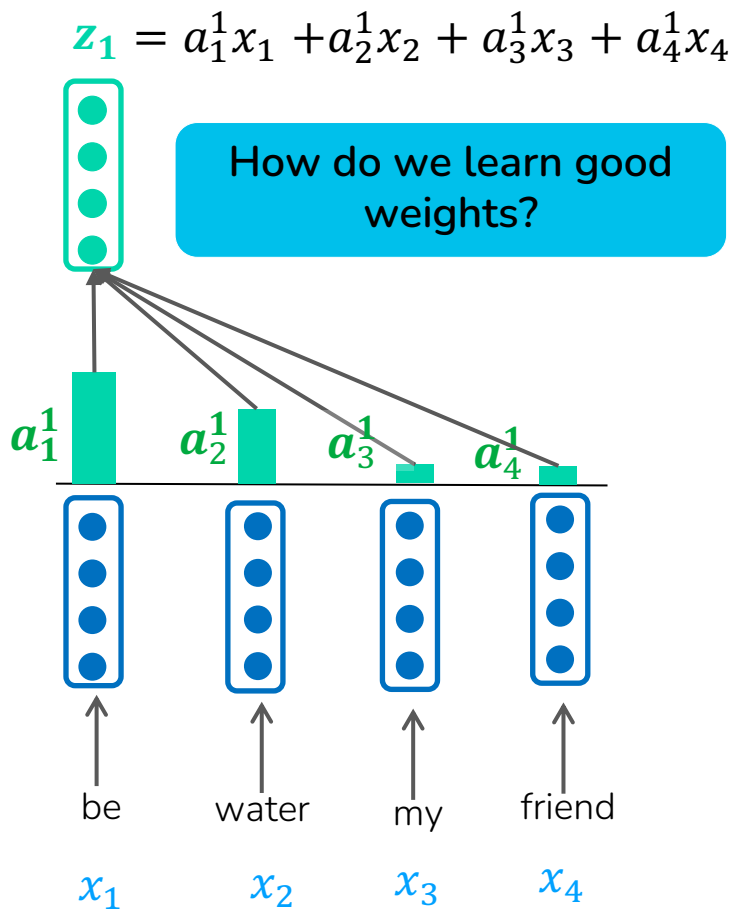
**Attention Is All You Need**

02.

Self-attention

# Self-attention lets a sequence attend to itself, all at once

- Main idea: “Translate” a sequence into itself and calculate attention scores between the tokens of the sequence
- How much do the other (or previous) tokens in the sequence influence a token?
- This **self-attention** score yields a powerful representation of each token that encapsulates the whole context in a **context vector**
- And: all computations can be performed in parallel



# Queries, keys and values make the weights learnable

Divide by  $\sqrt{\dim(k_i)} = 8$  for more stability

|        | Score                       | Weight                                 |
|--------|-----------------------------|----------------------------------------|
| be     | $s_1 = q_1 \cdot k_1 = 112$ | $a_1 = \text{softmax}(s_1 / 8) = 0.88$ |
| water  | $s_2 = q_1 \cdot k_2 = 96$  | $a_2 = 0.12$                           |
| my     | $s_3 = q_1 \cdot k_3 = 16$  | $a_3 = 0.00$                           |
| friend | $s_4 = q_1 \cdot k_4 = 8$   | $a_4 = 0.00$                           |

We need three weight matrices

$$W_q, q_i = W_q x_i$$

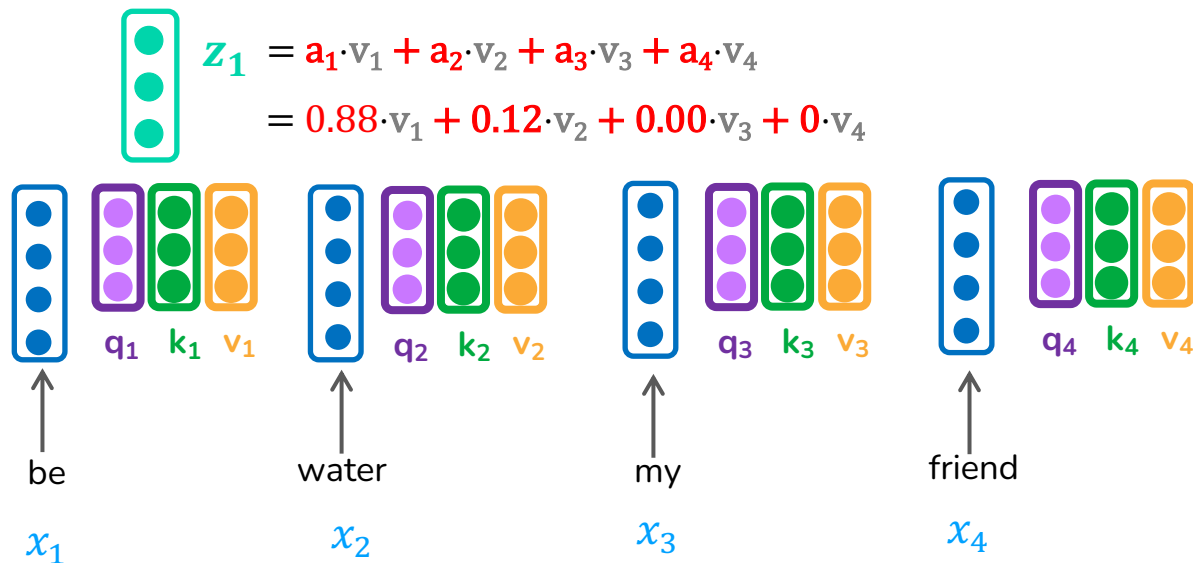
Queries

$$W_k, k_i = W_k x_i$$

Keys

$$W_v, v_i = W_v x_i$$

Values



# Dividing by $\sqrt{d_k}$

| Scores                | 112  | 96   | 16   | 8    |
|-----------------------|------|------|------|------|
| softmax, unscaled     | 1.00 | 0.00 | 0.00 | 0.00 |
| softmax, divided by 8 | 0.88 | 0.12 | 0.00 | 0.00 |

- Longer vectors give bigger dot products, and bigger scores push softmax towards a hard choice of one token.
- A hard choice has almost no gradient, so training slows down or stalls.
- Dividing by  $\sqrt{d_k}$  keeps the scores in a range where softmax still spreads its weight.

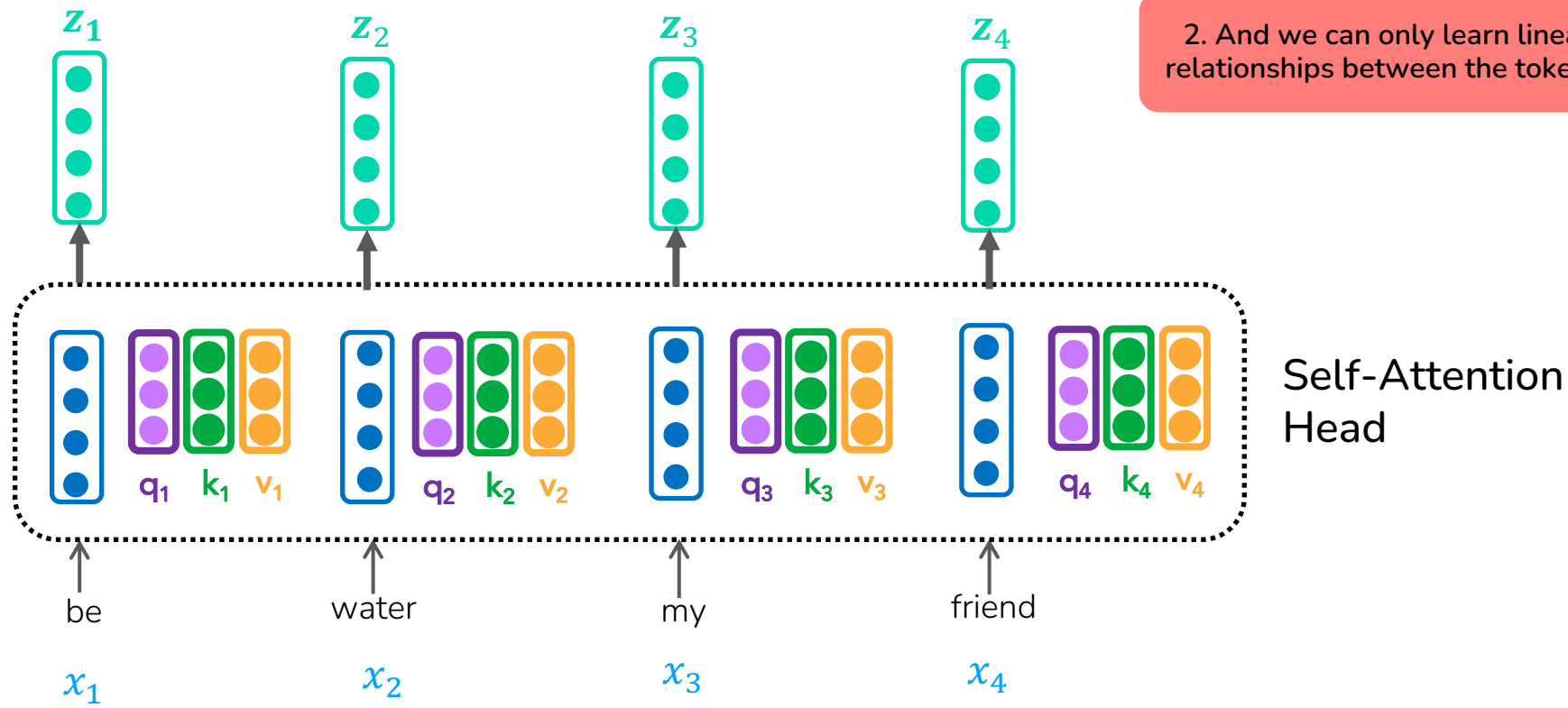
$$\textit{Attention}(Q, K, V) = \sigma(QK^T / \sqrt{d_k})V$$

Attention computes the company  
a word keeps.

# Self-Attention Head

1. But now we have no sense for the order of the tokens!

2. And we can only learn linear relationships between the tokens!



# Sources

## Literature

- Raschka, Build a Large Language Model (From Scratch), chapter 3, sections 3.3 and 3.4: self-attention without and with trainable weights, and the reason for dividing by  $\sqrt{d_k}$ . Used on the self-attention and scaling slides.
- Alammam, The Illustrated Transformer, [jalammar.github.io/illustrated-transformer](https://jalammar.github.io/illustrated-transformer). The worked example's 112 and 96, divided by 8. CC BY-NC-SA 4.0.
- Alammam, Visualizing A Neural Machine Translation Model. The animations on the attention decoder slides.
- Sutskever, Vinyals, Le 2014, Sequence to Sequence Learning with Neural Networks, [arxiv.org/abs/1409.3215](https://arxiv.org/abs/1409.3215).
- Bahdanau, Cho, Bengio 2014, Neural Machine Translation by Jointly Learning to Align and Translate, [arxiv.org/abs/1409.0473](https://arxiv.org/abs/1409.0473). The alignment matrix.

## Literature, continued

- Luong, Pham, Manning 2015, Effective Approaches to Attention-based Neural Machine Translation, [arxiv.org/abs/1508.04025](https://arxiv.org/abs/1508.04025).
- Hochreiter, Schmidhuber 1997, Long Short-Term Memory.
- Wu et al. 2016, Google's Neural Machine Translation System, [arxiv.org/abs/1609.08144](https://arxiv.org/abs/1609.08144).
- Vaswani et al. 2017, Attention Is All You Need, [arxiv.org/abs/1706.03762](https://arxiv.org/abs/1706.03762).
- Stanford CS224n, the vanishing gradient example.

# Sources, checked numbers

## Checked numbers, as of Oct 2, 2026

- The worked example: softmax of 112, 96, 16 and 8 divided by 8 gives 0.8808, 0.1192, 0.0000054 and 0.0000020, shown as 0.88, 0.12, 0.00 and 0.00. Own computation.
- The same scores unscaled: 0.9999999, 0.0000001, 0 and 0, shown as 1.00 and zeros. Own computation.

## Not used

- Last year's  $a_1 = .87$  and  $a_3 = .01$ : they do not match the slide's own scores.
- “LSTMs became popular in 2013 when Geoffrey Hinton joined Google”: the causal link is not supported.
- Our own model's head dimension: depends on the model configuration, still to be settled.